

08-Python Operators Lecture

Introduction

This summary covers a Python programming lecture focused on the concept of operators, including explanations, practical examples, and assignment instructions.

Categories of Operators in Python

- **Arithmetic Operators:**
addition (+), subtraction (-), multiplication (*), float division (/), floor division (//), modulo (%), exponent (**).
- **Comparison Operators:**
>, <, >=, <= (returns Boolean values, mostly for similar data types; exceptions are int and float).
- **Equality Operators:**
==, != (checks both value and, in most cases, type; exception for int and float).
- **Logical Operators:**
and, or, not (returns Boolean values, with clear truth table behaviors for combinations).
- **Bitwise Operators:** &, |, ^, ~, <<, >> (works only with integers; performs binary-level calculations).
- **Assignment Operators:** =, compound forms (+=, -=, *=, /=, //=, %=, **=, &=, |=, ^=, <<=, >>=).
- **Identity Operators:** is, is not (checks if operands refer to the same object in memory).
- **Membership Operators:** in, not in (checks for membership in sequence/collection types).
- **Ternary Operator:** `expr1 if condition else expr2` (inline conditional evaluations).

Arithmetic Operators Detailed

- Addition, subtraction, multiplication: straightforward numeric operations.
- **Float Division (/):** returns a float result (e.g., $100/3 \rightarrow 33.333\dots$)
- **Floor Division (//):**
returns the floor value of the division result (e.g., $100//3 \rightarrow 33$); if either operand is a float, returns float result.
- **Modulo (%):** returns the remainder of division.
- **Exponent ():** raises to the power (e.g., $10^3 \rightarrow 1000$).
- Usage of Python's `math` module is shown for advanced functions like `math.floor()`, `math.ceil()`.
- **Operator overloading:** '+' also concatenates strings and lists, '*' repeats sequences.
 - For concatenation, both operands must be of same type.
 - Operator returns a new object (does not mutate originals).

Comparison and Equality Operators

- Comparison operators return Boolean values for similar data types (except int and float can be compared together).
- Cannot compare complex numbers.
- String comparisons follow ASCII value order; use `ord()` to get a character's code point, `chr()` to convert value to character.

- Equality (`==`) and inequality (`!=`) operators check both value and type (except int/float type differences are relaxed);
- Works on other data types including lists.

Logical Operators

- `and`, `or`, `not` follow Boolean algebra with specific rules:
 - `and`: returns first false operand or second if both true
 - `or`: returns first true operand or second if both false
 - `not`: inverts value
- Operate on non-Boolean values (e.g., numbers and empty sequences)
- 0 (and equivalents) are treated as false; everything else is true
- Operator precedence: `not` > `and` > `or`

Bitwise Operators

- Operate only on integers (as only ints can be represented in binary)
- **Negation** (`~`): unary, result is `-num-1`
- **Left Shift** (`<<`): shifts bits to left, equivalent to multiplying by 2^n
- **Right Shift** (`>>`): divides (floor) by 2^n

- **Bitwise AND (&), OR (|), XOR (^):** performed on each binary bit
- Can use `bin()` function to view binary representation and manually apply bitwise logic

Assignment and Compound Assignment Operators

- Standard assignment (`=`)
- Compound assignments for each arithmetic and bitwise operator (`+=`, `-=`, etc.)
- Example: `a += 100` is equivalent to `a = a + 100`

Identity Operators

- `is`, `is not`: checks if two names point to the exact same object (not just equal value)
- Demonstrated with memory address using `id()`
- Python's object reusability: integers in range -5 to 256, standard literals and certain string cases are reused in memory
- Editors may behave differently due to optimizations (e.g., Thonny, Google Colab)

Membership Operators

- `in`, `not in`
: Checks if a value is present as an element in collections (lists, tuples, dictionaries, sets, strings, etc.)
- Returns Boolean results according to membership status

- Works with sequence and collection types

Ternary Operator

- Syntax: `expr1 if condition else expr2`
- Can be nested for multiple conditions

Operator Precedence

- Reflects the order in which operators are evaluated: parentheses > exponent > unary operators > arithmetic > bitwise > comparison > equality > logical (not > and > or)
- Usually does not need to be memorized; problem-solving with examples is encouraged

Practical Notes and Warnings

- Overloaded operators (`+`, `*`) only operate if operands are compatible types
- Editor-specific behavior (object reuse and memory addresses) can differ from pure Python behavior
- Logical operators handle non-Boolean types in a context-dependent way (e.g., empty list/string is false)

Assignment Submission Instructions

- Assignments must be submitted as IPYNB (Jupyter Notebook) files by uploading to a shared Google Drive folder
- Each student should create a personal folder for their submissions
- Specific due date and email/drive submission processes outlined

Q&A and Clarifications

- The instructor answers numerous student questions about operator behavior, Python internals (object memory addresses, type coercion rules), and assignment logistics
- Emphasizes hands-on practice and careful reading of error messages

End of summary.

Notes powered by [CraftNote](#)